

Deployment Guide

Local setup, Docker, cloud deployment and CI/CD pipeline

Deployment Documentation

Project: FitBuddy — AI-Powered Personal Fitness Coach

Overview

FitBuddy is deployed as a **monolithic application** where FastAPI serves both the backend API and the frontend static files. This simplifies deployment to a single process.

1. Local Development Setup

Prerequisites

Requirement	Version	Check Command
Python	3.9+	<code>python --version</code>
pip	Latest	<code>pip --version</code>
Git	Any	<code>git --version</code>
Gemini API Key	—	Get from AI Studio

Step-by-Step Setup

```
# Step 1: Navigate to project directory cd "path/to/fitbuddy" # Step 2: Install Python dependencies
cd backend pip install -r requirements.txt # Step 3: Set Gemini API Key # Windows PowerShell:
$env:GEMINI_API_KEY = "your-api-key-here" # Windows CMD: set GEMINI_API_KEY=your-api-key-here #
```

```
linux/macOS: export GEMINI_API_KEY="your-api-key-here" # Step 4: Start the server uvicorn main:app  
--reload --port 8000 # Step 5: Open in browser # http://localhost:8000
```

Start Script (Windows)

A `start.bat` file is included for convenience:

```
echo off cd backend uvicorn main:app --reload --port 8000 pause
```

2. Docker Containerization

Dockerfile

```
FROM python:3.11-slim WORKDIR /app # Copy backend files COPY backend/requirements.txt  
/requirements.txt RUN pip install --no-cache-dir -r requirements.txt # Copy application code COPY  
backend/ ./backend/ COPY frontend/ ./frontend/ # Set working directory to backend WORKDIR  
/app/backend # Expose port EXPOSE 8000 # Run the application CMD ["uvicorn", "main:app", "--host",  
"0.0.0.0", "--port", "8000"]
```

Docker Build & Run

```
# Build image docker build -t fitbuddy:latest . # Run container docker run -d \ -p 8000:8000 \ -e  
GEMINI_API_KEY="your-api-key-here" \ --name fitbuddy \ fitbuddy:latest # Access at:  
http://localhost:8000
```

Docker Compose

```
version: '3.8' services: fitbuddy: build: . ports: - "8000:8000" environment: GEMINI_API_KEY=${GEMINI_API_KEY} volumes: - ./data:/app/backend # Persist SQLite database restart:  
unless-stopped
```

3. Cloud Deployment Options

Option A: Render.com (Recommended for Free Hosting)

1. Push code to GitHub
2. Create account at render.com
3. New Web Service → Connect GitHub repo

4. Settings:
- **Build Command:** `pip install -r backend/requirements.txt`
 - **Start Command:** `cd backend && uvicorn main:app --host 0.0.0.0 --port $PORT`
5. Add Environment Variable: `GEMINI_API_KEY = your-key`
6. Deploy → Auto-deploys on every push

Option B: Railway

1. Connect GitHub repo to railway.app
2. Add `GEMINI_API_KEY` environment variable
3. Deploy — Railway auto-detects Python/FastAPI

Option C: Heroku

```
# Procfile web: cd backend && uvicorn main:app --host 0.0.0.0 --port $PORT # Deploy heroku login
heroku create fitbuddy-app heroku config:set GEMINI_API_KEY=your-key git push heroku main
```

Option D: Vercel (Frontend Only)

Vercel is best for static frontends. For full-stack, use Render or Railway.

For a static-only version:

- Build static HTML with hardcoded API URL
- Deploy `frontend/` to Vercel

4. CI/CD Pipeline (GitHub Actions)

```
# .github/workflows/deploy.yml name: Deploy FitBuddy on: push: branches: [main] jobs: test: runs-on:
ubuntu-latest steps: - uses: actions/checkout@v3 - name: Set up Python uses: actions/setup-python@v4
with: python-version: '3.11' - name: Install dependencies run: pip install -r
backend/requirements.txt - name: Run health check run: | cd backend uvicorn main:app & sleep 5 curl
http://localhost:8000/api/health deploy: needs: test runs-on: ubuntu-latest steps: - name: Deploy to
Render run: curl -X POST ${ secrets.RENDER_DEPLOY_HOOK }
```

5. Environment Variables Reference

Variable	Description	Required
----------	-------------	----------

GEMINI_API_KEY	Google Gemini API key	■ Yes
PORT	Server port (default: 8000)	Optional

6. Database Persistence

The SQLite database (`fitbuddy.db`) is created automatically in the `backend/` directory. For production:

- **Docker:** Mount a volume to persist the DB file between container restarts
- **Cloud:** Use PostgreSQL (modify connection string in `main.py`)
- **Backup:** Regular backup of `fitbuddy.db` file recommended

7. Performance Recommendations for Production

Area	Recommendation
Database	Migrate from SQLite to PostgreSQL
Rate Limiting	Add <code>slowapi</code> to prevent API abuse
Caching	Cache generated plans in Redis for 30 minutes
CORS	Restrict <code>allow_origins</code> to specific domains
HTTPS	Use NGINX reverse proxy with SSL certificate
Monitoring	Add Sentry for error tracking
API Keys	Use secrets manager (AWS Secrets Manager, Vault)

8. Project Structure for Deployment

```
fitbuddy/ ├── backend/ │   ├── main.py │   ├── requirements.txt │   └── fitbuddy.db ├── frontend/ │   ├── index.html │   ├── static/ │   │   ├── style.css │   │   └── app.js │   └── Dockerfile ├── docker-compose.yml ├── .github/ │   ├── workflows/ │   └── deploy.yml └── start.bat
```

